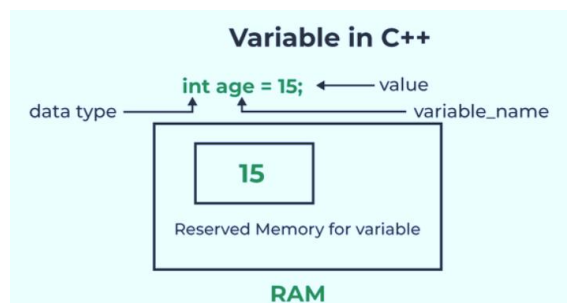


4-1 Variable Declaration

- Variables are containers for storing data values. A declaration is a process of naming the variables and their datatypes. This process informs the compiler about the existence and type of a variable. A variable is **an object** that may take on values of the specified type.
- Also , a variable is a location in the computer's memory where a value can be stored for later use by the program. Variables are like buckets that hold data. These data buckets are really locations in the computer's memory.



- The variable must be declared by specifying the datatype and the identifier as follows:
Datatype id.1, id2, ... , idn;
- A variable is defined by stating its type, followed by one or more spaces, followed by one or more variable names separated by commas, then followed by a semicolon. For example:

```
unsigned short int X;
int z;
float Y;
char A, a, c;
```

The primary differences between `int` and `short` `int` in C++ are their **size in memory** and the **range of values** they can store.

- Note:** C++ does distinguish between the above **A** and **a** variable (C++ is case-sensitive).

Datatype	Size in memory	Description
int	4 bytes	Stores whole numbers (positive, negative, or zero)
unsigned int	4 bytes	store non-negative, whole numbers (zero and positive integers).
char	1 byte	Stores a single character (like 'A', 'b', or '5').
float	4 bytes	Stores numbers with a fractional part.
double	8 bytes	Stores decimal numbers with more precision than float

✚ The C++ language provides the **sizeof()** operator to check the size of the data types.

```
#include <iostream>
using namespace std;
int main()
{
    // Use sizeof() to know size the data types
    cout<<"The size of int in byte:"<< sizeof(int)<<endl;
    cout<<"The size of char in byte:"<< sizeof(char)<<endl;
    cout<<"The size of float in byte:"<< sizeof(float)<<endl;
    cout<<"The size of double in byte: "<< sizeof(double);
    return 0;
}
```

sizeof() operator to check the size of the data types.

Example (1) The following program computes the arithmetic operators (+,-,/,*)

```
#include<iostream>
using namespace std;
int main()
{
    int a,b, sum, sub, mul, div;
    cout << "enter any two numbers"<<endl;
    cin>> a>>b;
    sum=a+b;
    sub=a-b;
    mul=a*b;
    div=a/b;
    cout<<"number1="<<a<<endl<<"number2="<<b <<endl<<"summation="<<sum<<endl;
    cout<<"subtraction="<<sub<<endl;
    cout<<"multiplication="<<mul<<endl;
    cout<< "division="<< div<< endl;

    return 0;
}
```

Example (2) Write a program that reads the radius of a circle, then computes and outputs its area.

```
#include<iostream>
using namespace std;
int main()
{
    const float pi = 3.14;
    int radius;
    float area;
    cout << "enter the radius of circle:";
    cin>>radius;
    cout<<endl;
    area = radius * radius * pi;
    cout <<"the area of circle:"<< area;
    return 0;
}
```

Output:

enter the radius of circle:2
the area of circle:12.56

Example 3: Write the following equation as a C++ expression:

$$f = \frac{a+b+c+d+e}{10}$$

Sol.

```
float f;
int a,b,c,d,e;
f=(a+b+c+d+e)/10;
```

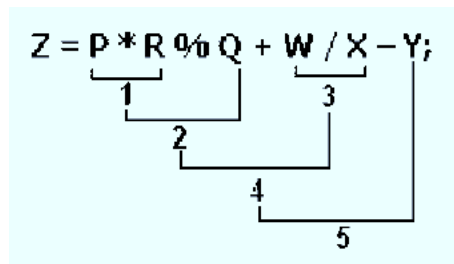
Note: the parentheses here are required because division has higher precedence than addition.

Example 4: State the order of evaluation for the following expression:

$$Z = P * R \% Q + W / X - Y;$$

Sol.

The order	The operator
1	*
2	%
3	/
4	+
5	-



Example 5: Write C++ program to perform the equation at example 4:

```
#include<iostream>
using namespace std;
int main()
{
    int Z, P, R, Q, W, X, Y;
    cout << "enter P:"; cin >> P;
    cout << "enter R:"; cin >> R;
    cout << "enter Q:"; cin >> Q;
    cout << "enter W:"; cin >> W;
    cout << "enter X:"; cin >> X;
    cout << "enter Y:"; cin >> Y;
    Z= P * R % Q + W / X - Y;
    cout << "the result=" << Z;

    return 0;
}
```

4-2 math library

The "math.h" library contains the common mathematical functions used in scientific equations.

Mathematical Expression	C++ Expression
e^n	Exp(x)
$\text{Log}(x)$	$\text{Log}_{10}(x)$
$\text{Ln}(x)$	Log(x)
$\text{Sin}(x)$	Sin(x)
X^n	Pow(x,n)
$\sqrt{x}$	Sqrt(x)

Example (1):

Write the following equation as a C++ expression and state the order of evaluation of the operators:

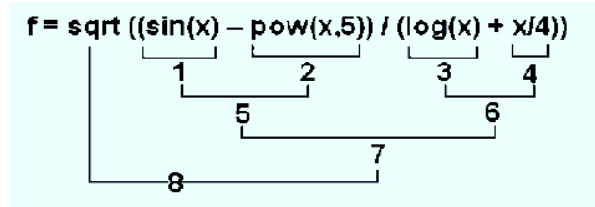
$$f = \sqrt{\frac{\sin(x) - X^5}{\ln(x) + \frac{x}{4}}}$$

Sol.

C++ code will be as:

```
f = sqrt ((sin(x)-pow(x,5)) / (log(x) + x/4))
```

Order of evaluation:



Homework:

Write the following equation as a C++ expression and state the order of the operators.

$$z = \sqrt{\frac{x^2 y - 3 \sin(x)}{\tan(x^3) + \frac{x^3}{y}}}$$

4-3 Increment and Decrement Operators

Increment Operator (++)

The increment operator (++) increments the value of a variable by 1. The increment can be done in two ways:

A. Prefix Increment

In this method, the operator precedes the operand (e.g., ++a). The value of the operand will be altered *before* it is used. For example:

```
int a = 1;
int b = ++a;    // b = 2 ← this the output
```

B. Postfix Increment

In this method, the operator follows the operand (e.g., a++). The value operand will be altered *after* it is used. For example:

```
int a = 1;
int b = a++;    // b = 1 ← this the output
int c = a;      // c = 2 ← this the output
```

Below example shows the implementation of **increment (++)**:

```
#include<iostream>
using namespace std;
int main()
{
    int a = 1;    int b = 3;
    cout<<"Pre-Incrementing a ="<< ++a <<endl;
    cout<<"Post-Incrementing b ="<< b++<<endl;
    cout<<"New-Value after Incrementing b ="<< b;
    return 0;
}
```

Output:

Pre-Incrementing a =2

Post-Incrementing b =3

New-Value after Incrementing b =4

Decrement Operator (--)

The **decrement operator (--)** is used to decrement the value of the variable by 1. The decrement can be done in two ways:

A. Prefix Decrement

In this method, the operator precedes the operand (e.g., --a). The value of the operand will be altered *before* it is used. For example:

```
int a = 1;
int b = --a; // b = 0
```

B. Postfix Decrement

In this method, the operator follows the operand (e.g., a--). The value of the operand will be altered *after* it is used. For example:

```
int a = 1;
int b = a--; // b = 1
int c = a; // c = 0
```

Below example shows the implementation of the **decrement operator**

```
#include <iostream>
using namespace std;
int main() {
    int a = 1;
    int b = 1;
    cout<<"Pre-Decrementing a="<< --a<<endl;
    cout<<"Post-Decrementing b="<<b--<<endl;
    cout<<"After-Decrementing b="<<b--<<endl;
    return 0;
}
```

Output

```
Pre-Decrementing a =0
Post-Decrementing b=1
After-Decrementing b=0
```

4-4 Selection statement

Conditional expressions are mainly used for decision making. C++ provides multiple selection structures: **if**, **if/else**, **else if**, **nested if** and **switch**.

Single if statement structure

The **if** in C++ is the simplest decision-making statement. It consists of the test condition and a block of code that is executed if and only if the given condition is true. Otherwise, it is skipped from execution.

Syntax of if in C

```
if (condition)
{
    // Statements to execute if condition is true
}
```

Example(1):

```
if (average >=70)
{
    cout << "good";
}
```

Example (2):

Write program to check whether the number is negative by comparing it with 0 and checking if it is less than zero, it is negative.

Sol.

```
#include <iostream>
using namespace std;
int main() {
    int n = -4;

    // Condition to check for negative number
    if (n < 0)
    {
        cout<<"yes, it is Negative: "<<n ;
    }
    return 0;
}
```

Output:

yes, it is Negative: -4

Example (3):

Write program to check whether the number is in the range [10:20].

Sol.

Here we will need to combine two conditions. One to check for upper limit and one to check for lower limit.

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout<< "please enter the number n : ";
    cin>>n;
    // Check whether n is the range [10, 20]
    if (n <= 20 && n >= 10)
    {
        cout<<n<< ":is in the range [10, 20]";
    }
    return 0;
}
```

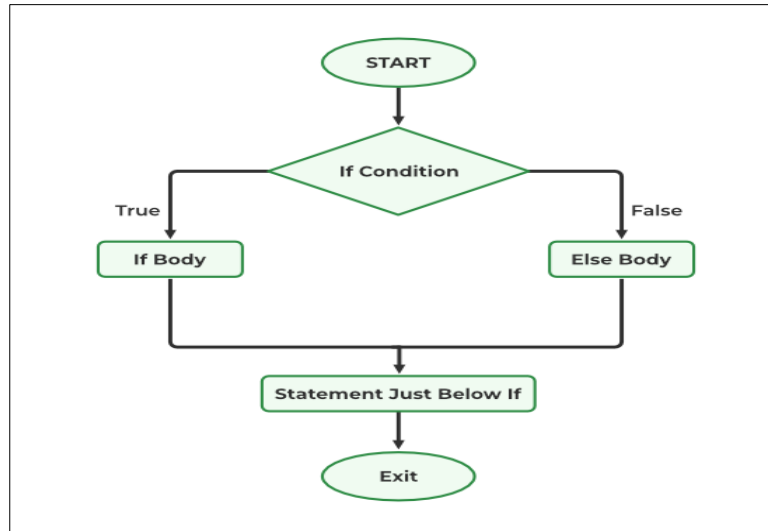
Output:

please enter the number n : 17

17: is in the range [10, 20]

if-else Statement Structure

The **if-else** in C++ is an extension of the if statement, which not only allows the program to execute one block of code if a condition is true, but also a different block if the condition is false. If the condition is true, then the code inside the if block is executed, otherwise the code inside the else block is executed. The flowchart below explains the if-else works in C++:



Example (4)

Write a C++ program to read a student's degree, and check if the degree is greater than or equal to 50, then print **passed the exam**, otherwise print **failed the exam**:

```
#include<iostream>
using namespace std;
int main()
{
    int degree;
    cout<<"input the degree, please : ";
    cin >>degree;
    if (degree>=50 )
    {
        cout << "the student passed the exam";
    }
    else
    {
        cout<< "the student failed in the exam";
    }

    return 0;
}
```

Output:
input the degree, please: 75
the student passed the exam